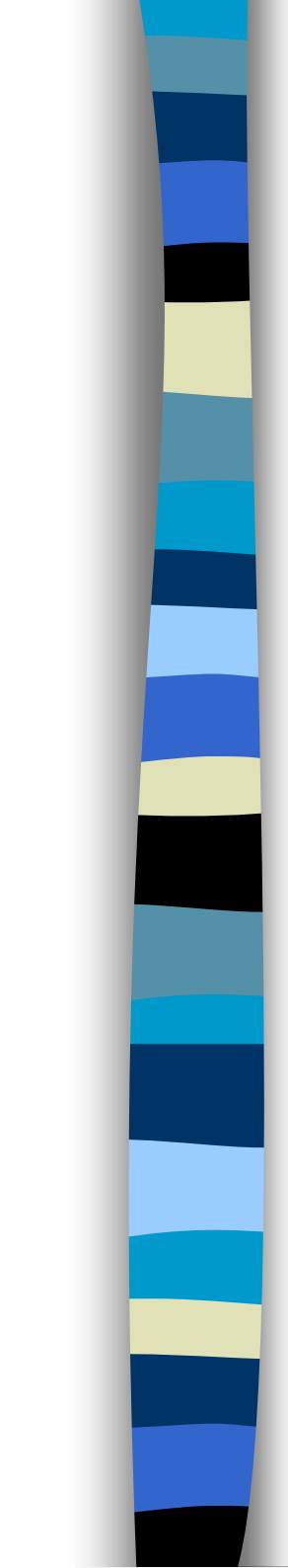


Programação Orientada a Objetos

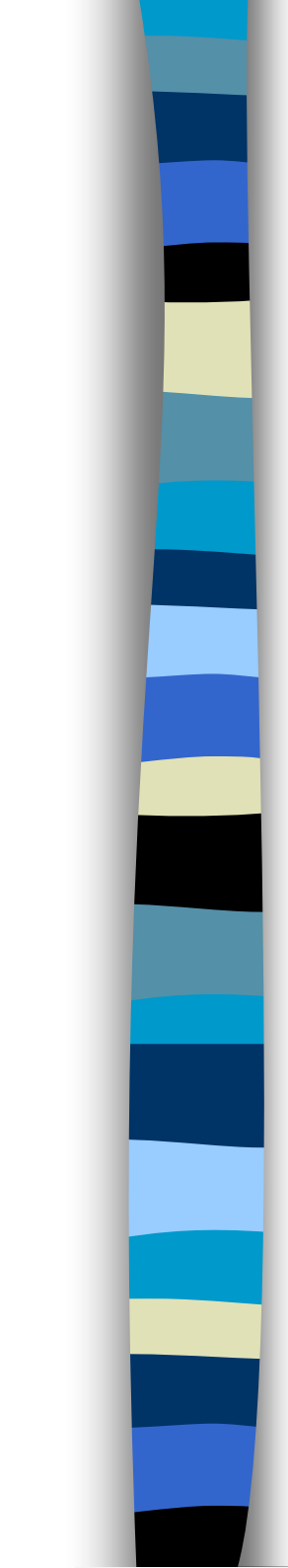
- Introdução -



➤ Evolução das Abordagens de Desenvolvimento de Sistemas

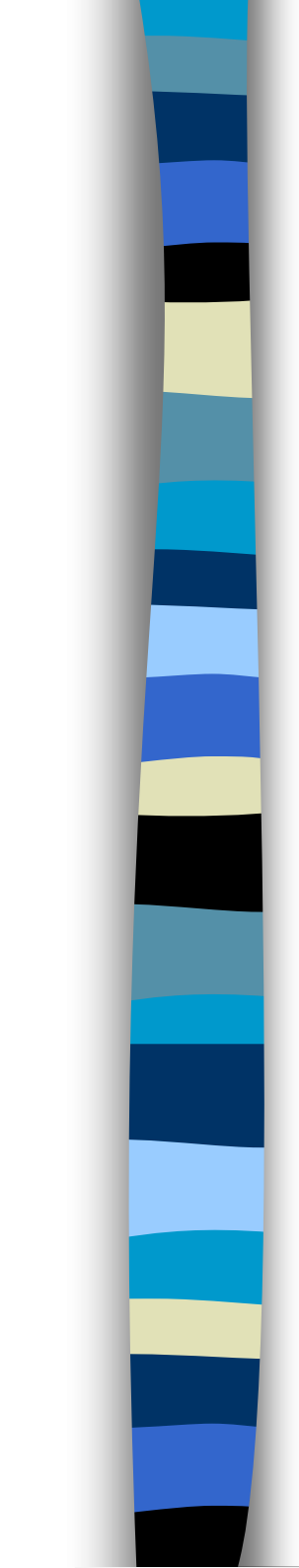
Monolítico

- Década de 60
- Aplicações/Sistemas eram pequenos e relativamente simples
- Linguagens simples (Fortran, Cobol)
- Os desenvolvedores não tinham um método para desenvolver sistemas, utilizavam de sua criatividade pessoal
- A programação era considerada uma atividade artesanal e criativa
- Os códigos do tipo **Spaghetti** e de difícil manutenção



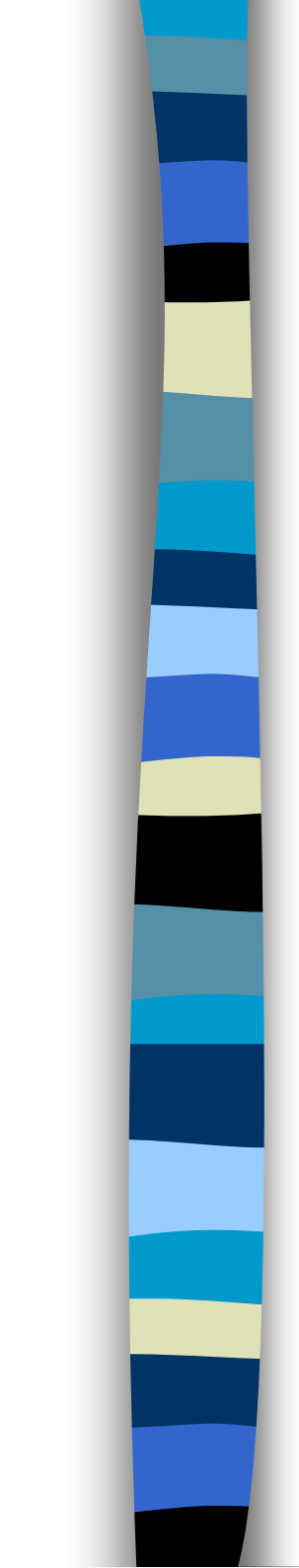
Estruturado

- Década de 70
- Al Constantine e Yourdon criaram uma abordagem de desenvolvimento de sistemas que utilizava a função como seu bloco de construção. Este método foi denominado **análise e desenho estruturado**.
- Surgiu a **programação modular** que facilitou a manutenção dos sistemas



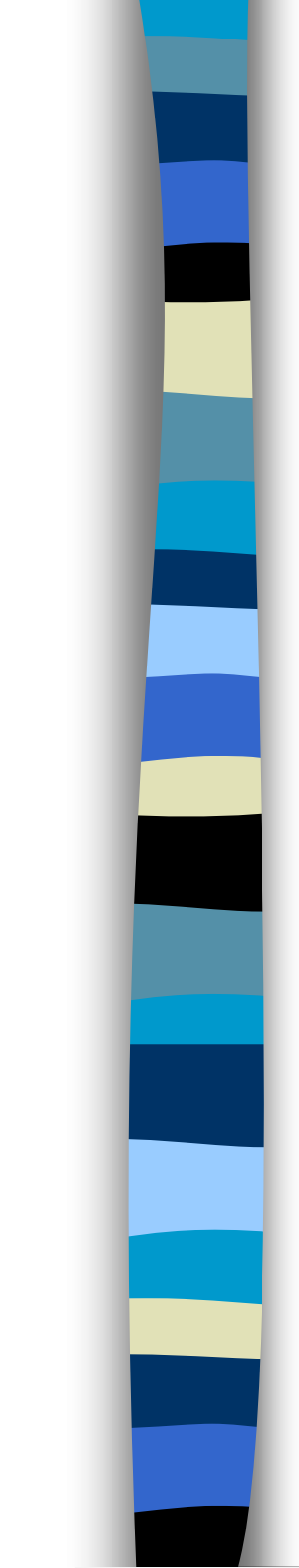
Estruturado

- Utilização de diagramas gráficos para representar os fluxos de dados (**DFD**) e para representar as lógicas de programação (**Fluxogramas**)
- Década de 80, Peter Chen (criador do diagrama de **entidade e relacionamentos**) e Ed Codd (desenhista do banco de dados relacional) criando um método (Modelagem de Dados) de desenvolvimento de sistemas baseados em dados



Orientado a Objetos

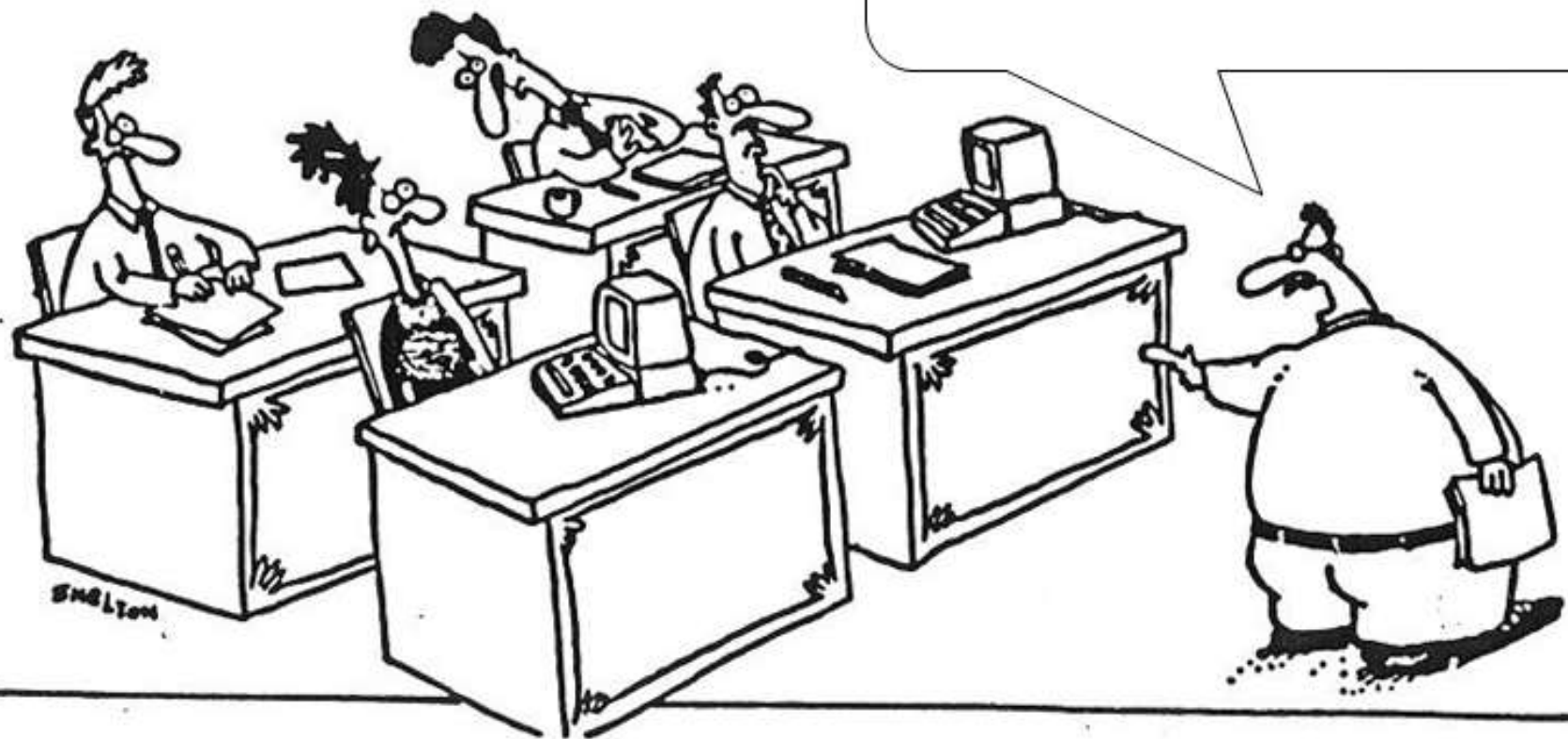
- Década de 90
- Visa complementar o método estruturado
- Fornece ao desenvolvedor uma **perspectiva ortogonal**, permitindo administrar a complexidade do domínio do problema e sua tecnologia de suporte
- Facilita a administração dos **dados** e o controle das funções (**código**)
- Permite a produção de software mais flexível e de fácil manutenção



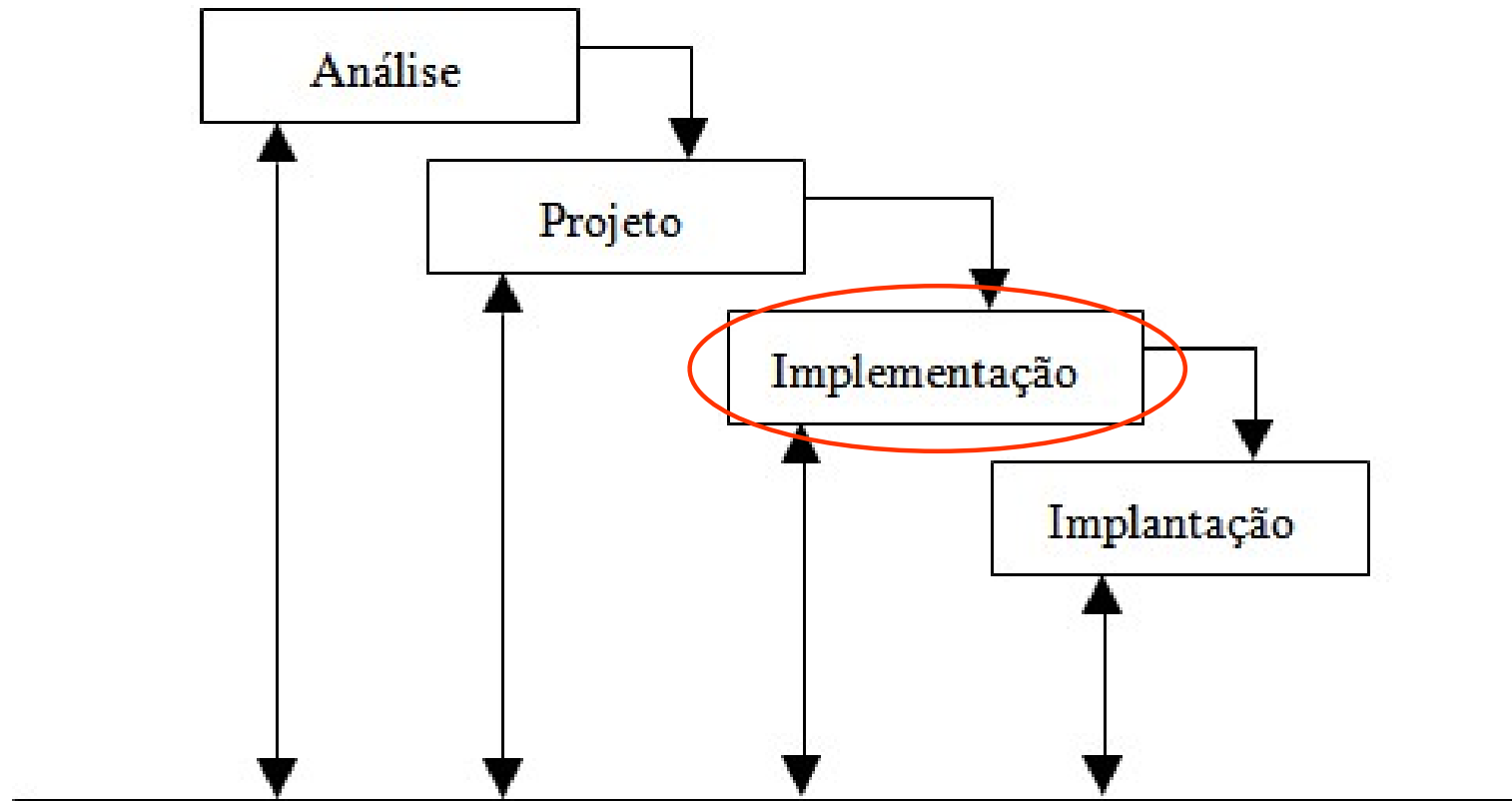
Programação Orientada a Objetos

- A **Complexidade** faz parte da **essência** do software
- Devemos utilizar de **mecanismos** que permitem **administrar** a **complexidade** do software
- A **complexidade** gera **dificuldades**
- Muitas **dificuldades** encontradas no desenvolvimento de um software podem surgir devido a forma empregada para construí-lo

Eu vou lá em cima descobrir
o que eles querem...
E vocês aí comecem a codificar!



Etapas para o Desenvolvimento de uma Aplicação



Abordagem O.O.

- Linguagem de Especificação UML -

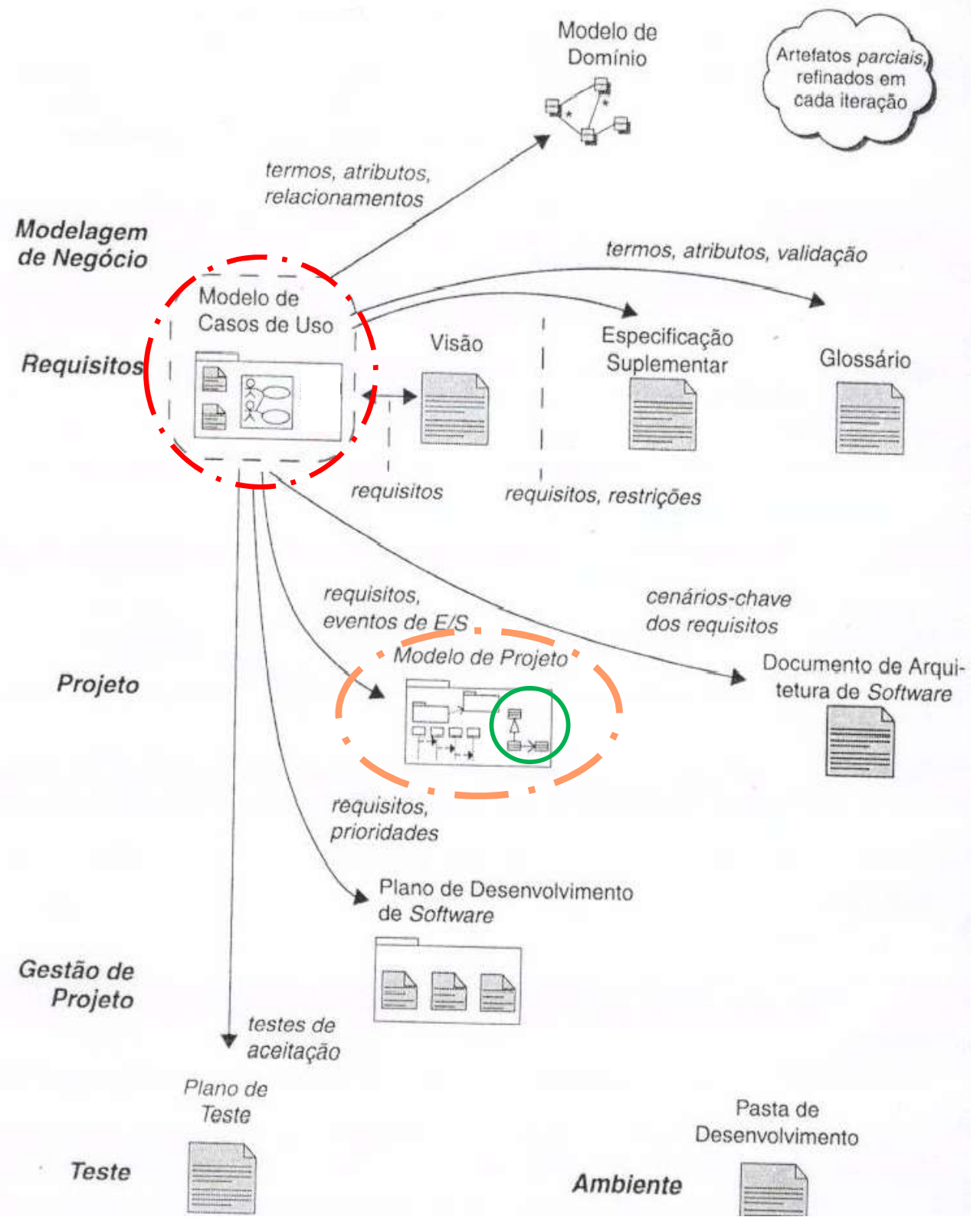
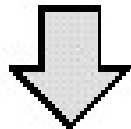


Figura extraída do Livro Utilizando UML e Padrões
- Graig Lerman - 2a. Edição -

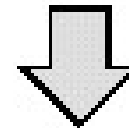
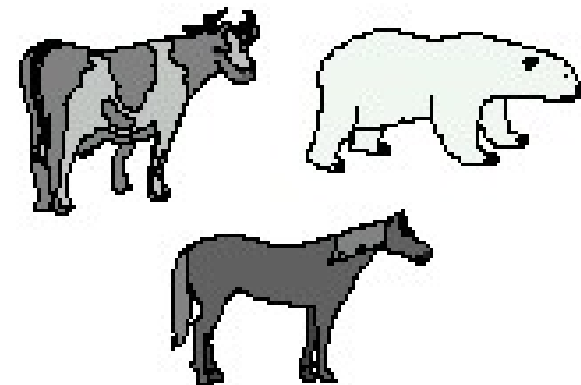
Fundamentos da Programação Orientada a Objetos

Classe

Uma **classe** representa um conjunto de **elementos** (**objetos**) que possuem as mesmas propriedades (**atributos**) e as mesmas **operações** (**métodos**).



Aeronave



Mamífero

- Os **atributos** de uma classe representa uma **estrutura de dados** e os **métodos** representam um **código** definido a partir de uma **lógica de programação**.
- A representação de uma classe é feita a partir da seguinte simbologia .

Nome da classe

Atributo

atributo: tipo do dado

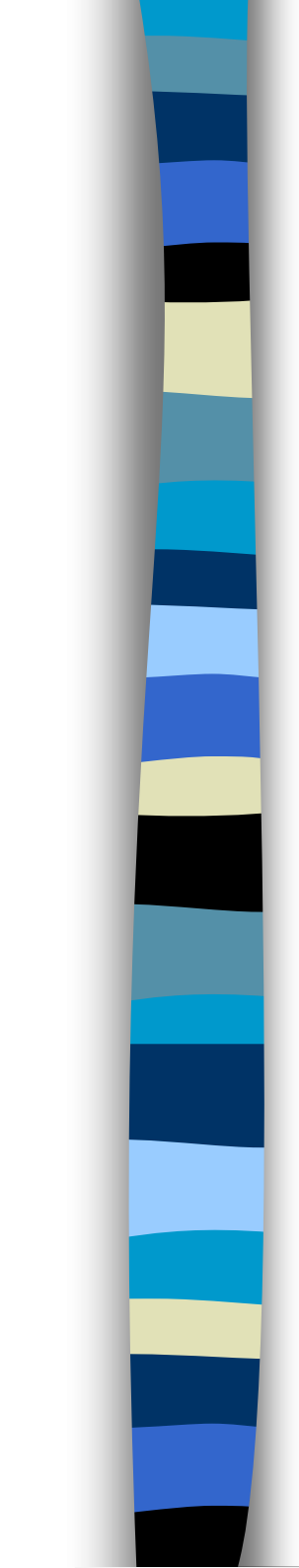
atributo: tipo do dado = valor inicial

...

Operação

Operação (lista de argumentos): tipo do resultado

...

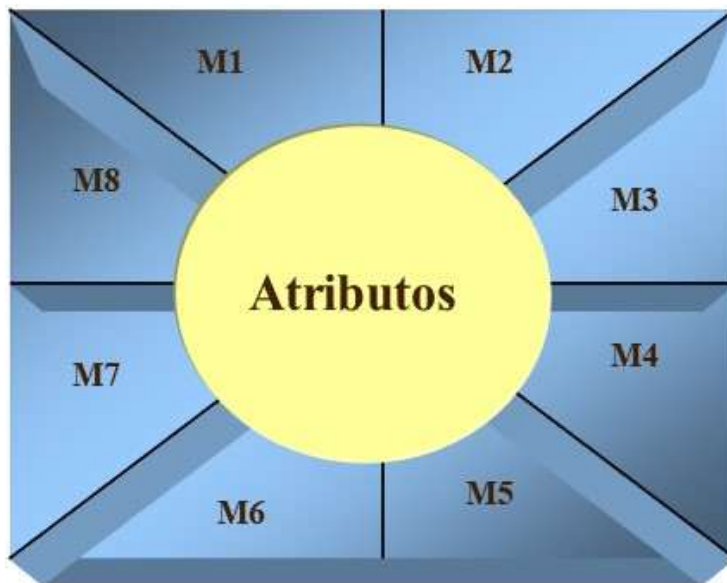


Definição dos Atributos: Todos os objetos de uma classe terão o mesmos **atributos**. Cada atributo representa um tipo de dado (inteiro, real, caracter, matriz, ponteiro, etc ...). A diferença entre os objetos de uma mesma classe é representada pelo armazenamento de diferentes valores para estes seus atributos.

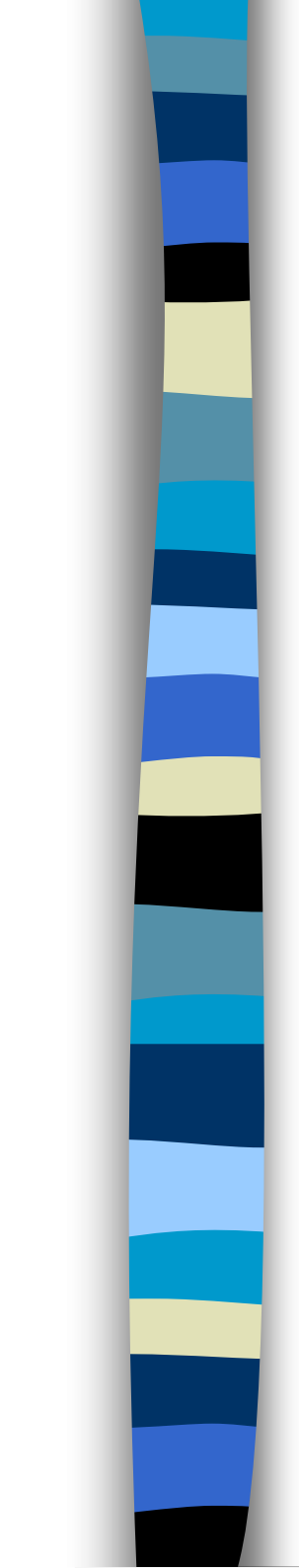
Definição dos Métodos: Todo método é um bloco de instruções cuja finalidade é permitir que o objeto execute uma determinada operação pré-estabelecida. Os objetos instanciados a partir de uma mesma classe utilizam os mesmos **métodos** (código) e **somente os métodos do próprio objeto terão acesso sobre os seus atributos.**

Objeto

Representa um elemento (instância) de uma determinada classe. Objetos de uma mesma classe possuem os mesmos **atributos** e realizam as mesmas **operações**. Dentro da orientação a objetos as operações realizadas por um objeto são denominadas de **métodos**.



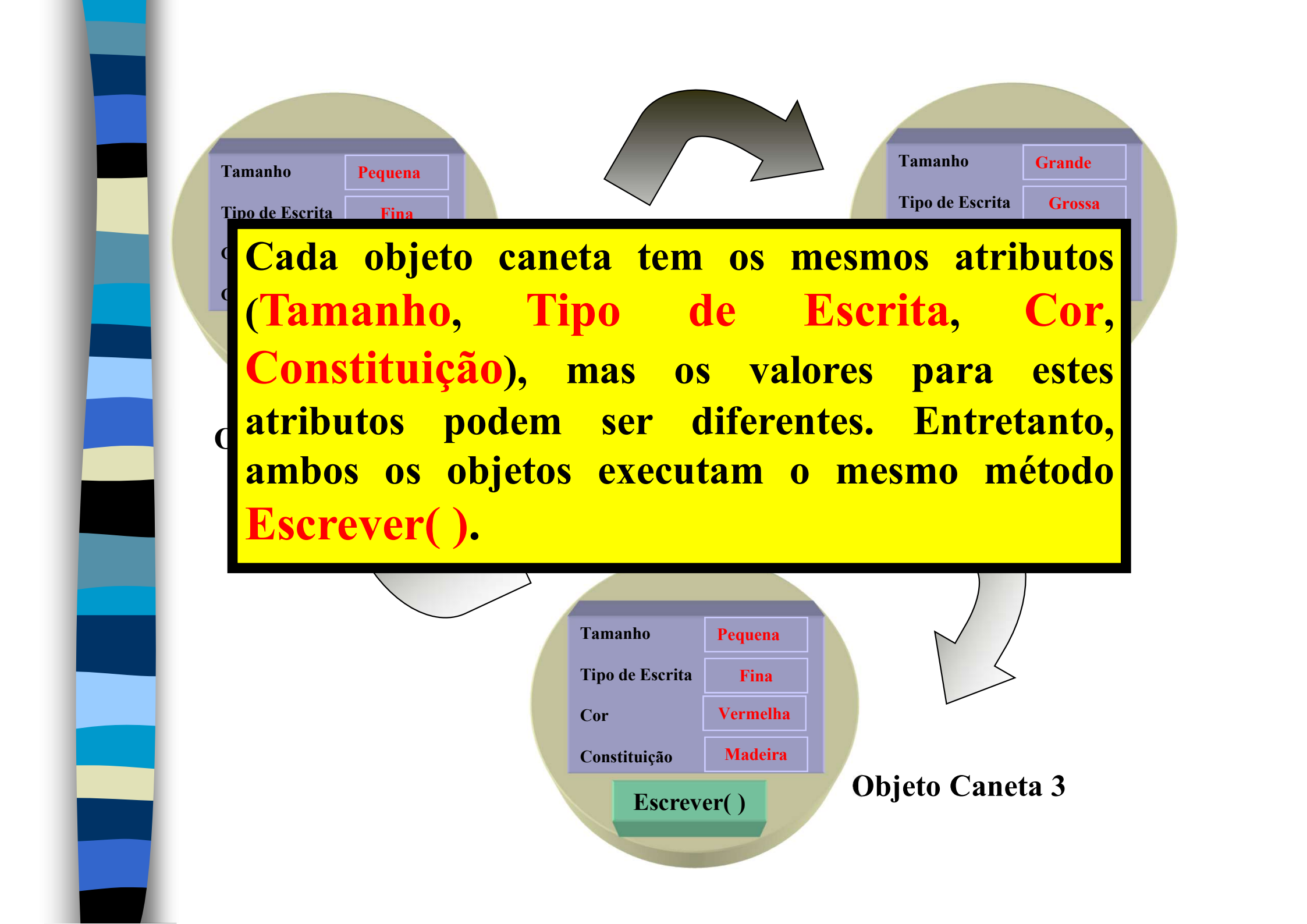
M1, M2, M3, M4, M5, M6, M7, M8
(Métodos -Código)



Classe/Instância/Objeto

Instanciação

Representa o processo de criação de um objeto a partir de uma determinada classe.



Tamanho	Pequena
Tipo de Escrita	Fina

Tamanho	Grande
Tipo de Escrita	Grossa

Cada objeto caneta tem os mesmos atributos (**Tamanho**, **Tipo de Escrita**, **Cor**, **Constituição**), mas os valores para estes atributos podem ser diferentes. Entretanto, ambos os objetos executam o mesmo método **Escrever()**.

Tamanho	Pequena
Tipo de Escrita	Fina
Cor	Vermelha
Constituição	Madeira

Escrever()

Objeto Caneta 3

Representação de uma classe através da UML



Figura Geométrica
Retângulo

Características

Altura

Base

Operações

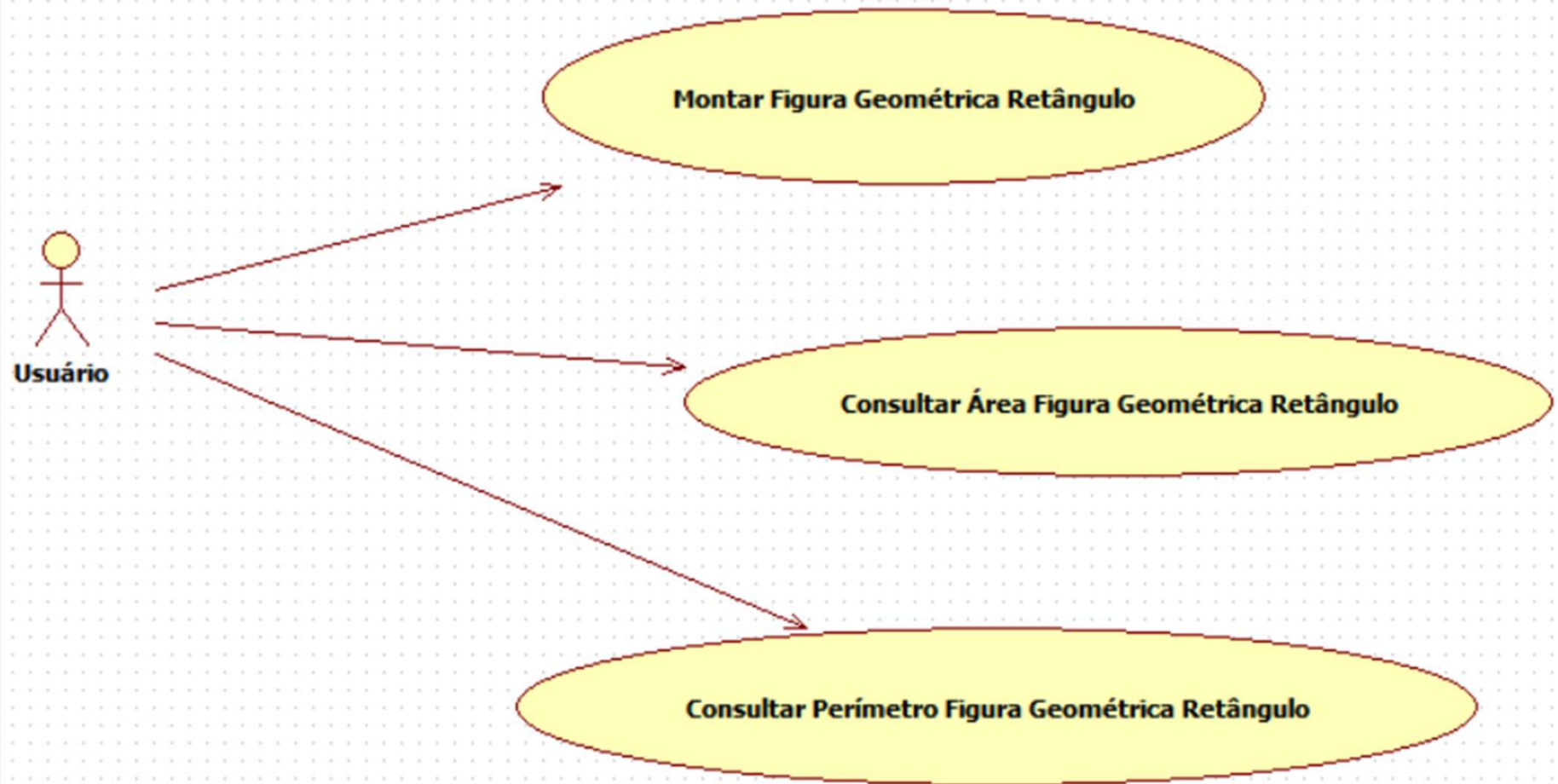
Apontar Altura

Apontar Base

Calcular Área

Calcular Perímetro

Representação UML – Diagrama de Casos de Uso



Representação UML – Diagrama de Casos de Uso

Nome do Caso de Uso	Montar Figura Geométrica Retângulo
Ator Principal	Usuário
Atores secundários	
Pré-Condições	Nenhuma
Pós-Condições	Nenhuma
Ações do ator	Ações do Sistema
1. Informa a medida da altura da figura geométrica retângulo	
2. Informa a medida da base da figura geométrica retângulo	
	3. Monta uma representação computacional de uma figura geométrica retângulo a partir do apontamento das medidas da altura e da base
Restrições/Validações	Nenhuma

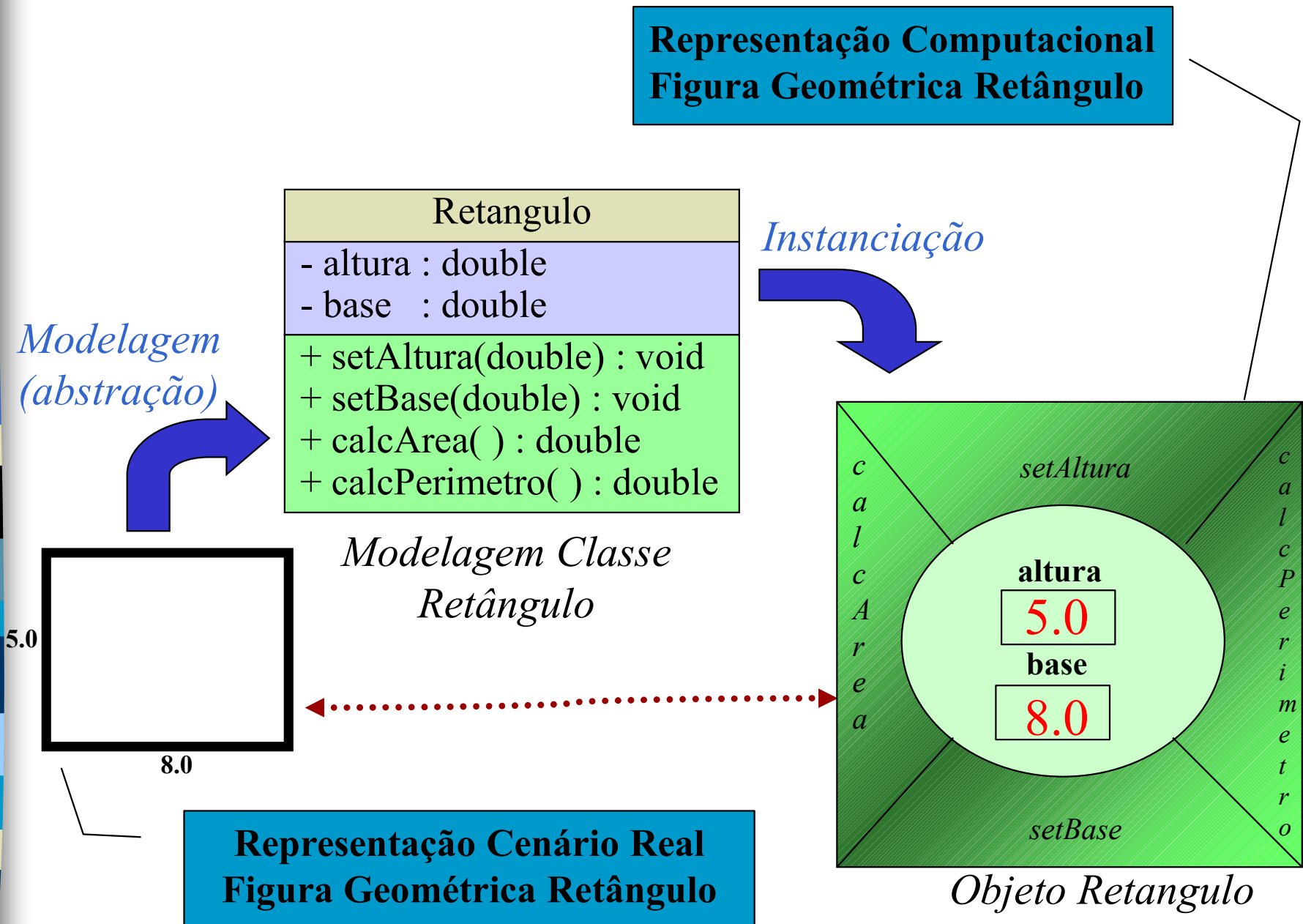
Representação UML – Diagrama de Casos de Uso

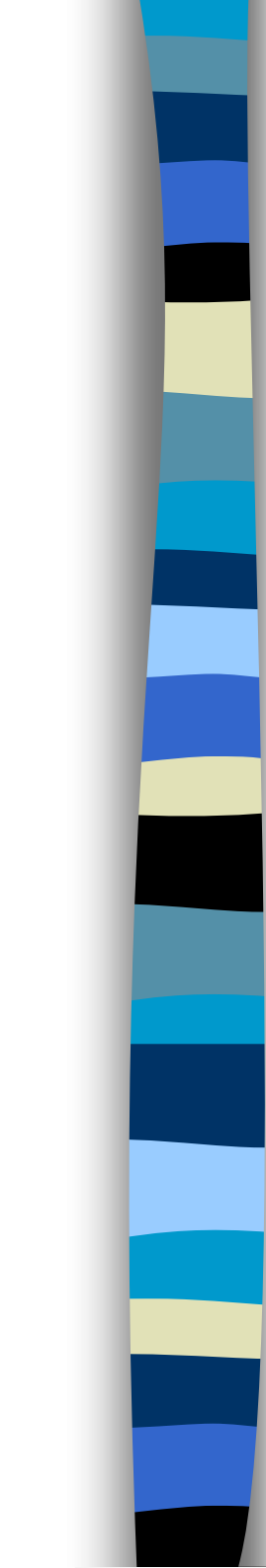
Nome do Caso de Uso	Consultar Área Figura Geométrica Retângulo
Ator Principal	Usuário
Atores secundários	
Pré-Condições	Ter montado a figura geométrica retângulo
Pós-Condições	Nenhuma
Ações do ator	Ações do Sistema
1. Solicita a área da figura geométrica retângulo	
	2. Realiza o cálculo da figura geométrica retângulo a partir da execução da operação de cálculo de área
	3. Devolve o resultado do cálculo
Restrições/Validações	Nenhuma

Representação UML – Diagrama de Casos de Uso

Nome do Caso de Uso	Consultar Perímetro Figura Geométrica Retângulo
Ator Principal	Usuário
Atores secundários	
Pré-Condições	Ter montado a figura geométrica retângulo
Pós-Condições	Nenhuma
Ações do ator	Ações do Sistema
1. Solicita o perímetro da figura geométrica retângulo	
	2. Realiza o cálculo da figura geométrica retângulo a partir da execução da operação de cálculo de perímetro
	3. Devolve o resultado do cálculo
Restrições/Validações	Nenhuma

Representação UML – Diagrama de Classes





Retangulo
- altura : double - base : double
+ setAltura(double) : void + setBase(double) : void + calcArea() : double + calcPerimetro() : double

- O método **setAtributo** tem a finalidade de obter um determinado dado e armazená-lo no respectivo atributo indicado pelo método
- Modo de Acesso Privado (private)
 - **altura : double** o símbolo - indica que somente os métodos definidos na classe tem acesso direto ao atributo.
- Modo de Acesso Público (public)
 - + **setAltura(double) : void** o símbolo + indica que a execução do método pode ser solicitada pela aplicação ou por outro objeto a partir do **Mecanismo de Passagem de Mensagens**.

Retangulo

- altura : double

- base : double

+ setAltura(double) : void

+ setBase(double) : void

+ getAltura() : double

+ getBase() : double

+ calcArea() : double

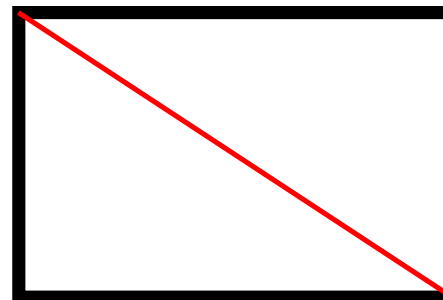
+ calcPerimetro() : double

- Faça a implementação dos métodos getAltura e getBase. O método getAtributo tem a finalidade de retornar o conteúdo do atributo indicado pelo método.

Retângulo
- altura : double - base : double
+ setAltura(double) : void + setBase(double) : void + getAltura() : double + getBase() : double + calcArea() : double + calcPerimetro() : double + calcDiagonal() : double

- Faça a implementação do método calcDiagonal. O método tem a finalidade de calcular e retornar o valor da diagonal do retângulo. Utilize o teorema de Pitágoras.

Figura Geométrica Retângulo



Exercícios

1) Faça a implementação da classe Circulo na linguagem Java e em seguida faça a instanciação de um objeto da respectiva classe e ao final faça a interação com o objeto instanciado. Utilize a classe Scanner para realizar a entrada de dados.

Circulo
- raio : double
+ setRaio(double) : void + getRaio() : double + calcArea() : double + calcPerimetro() : double + calcDiametro():double

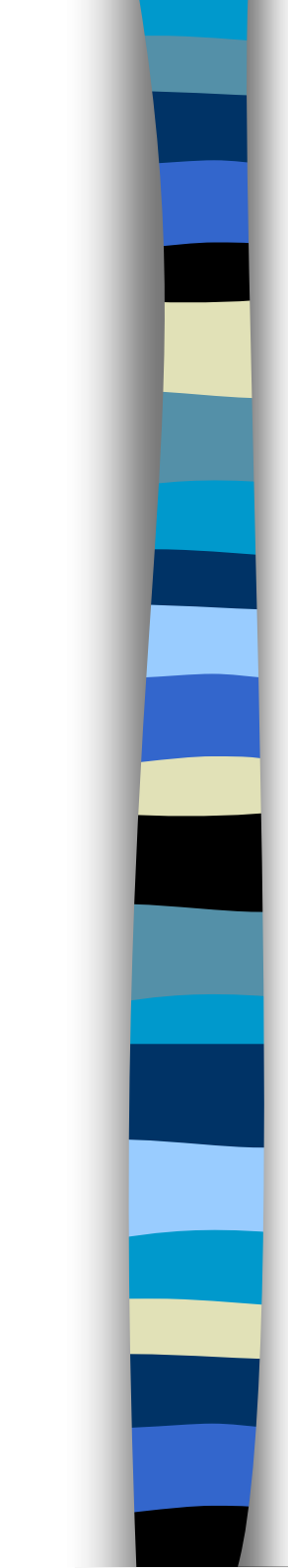
Calcula e devolve a medida da área. Onde área é πr^2

Calcula e devolve a medida do perímetro. Onde perímetro é $2\pi r$

Calcula e devolve a medida do diâmetro. Onde diâmetro é $2r$

2) Faça a implementação da classe Aluno na linguagem Java e em seguida faça a instanciação de um objeto da respectiva classe e ao final faça a interação com o objeto instanciado. Utilize a classe Scanner para realizar a entrada de dados.

Aluno	
- RA : int	Registro Acadêmico do Aluno
- NtPrv1 : double	Nota da primeira prova
- NtPrv2 : double	Nota da segunda prova
- NtTrab1 : double	Nota do primeiro trabalho
- NtTrab2 : double	Nota do segundo trabalho
+ setRA(int) : void + setNtPrv1(double) : void + setNtPrv2(double) : void + setNtTrab1(double) : void + setNtTrab2(double) : void + getRA() : int + getNtPrv1() : double + getNtPrv2() : double + getNtTrab1() : double + getNtTrab2() : double + calcMediaProva() : double	
	Calcula e devolve a média das provas. O cálculo da média das provas deve se calculado da seguinte forma: $0.75 * (\text{Nota da Primeira Prova} + 2 * \text{Nota da Segunda Prova})/3$
+ calcMediaTrab() : double	Calcula e devolve a média dos Trabalhos. O cálculo da média dos trabalhos deve se calculado da seguinte forma: $0.25 * (\text{Nota do Primeiro Trabalho} + \text{Nota do Segundo Trabalho})/2$
+ calcMediaFinal() : double	Calcula e devolve a média do aluno. O cálculo da média do aluno é calculado da seguinte forma: Média da Prova + Média do Trabalho



Faça a saída de dados de acordo com a definição do menu abaixo:

- 1 - Exibir Nota das Provas/Trabalhos
- 2 - Exibir Média dos Trabalhos/Provas
- 3 - Exibir Média Final
- 4 - Sair

Digite a opcao:

Para cada uma das opções de 1 a 3 deve ser exibido o Ra do aluno.